EcoTrack · EmissionsTracker — Entrega completa (documento autocontenido)

Repositorio: https://github.com/Josuehmz/ecotrack-emissions-tracker (público). Este PDF reúne en un solo archivo todo lo que el enunciado pide, para poder evaluarlo sin abrir el repositorio: (1) guía de corrección (README), (2) bitácora de prompts iterativos con la evolución del código, (3) código fuente completo de los componentes, el hook y la configuración de Supabase, (4) capturas de la interfaz funcionando y (5) salida real de la verificación.

Índice

Parte 1 — README: entregables y rúbrica → evidencia
Parte 2 — Bitácora de prompts (4 prompts, versiones v1 a v4)
Parte 3 — Código fuente (EmissionsTracker.tsx, useEmissions.ts, Supabase…)
Parte 4 — Capturas (escritorio y móvil: carga, error, formulario, lista)
Parte 5 — Verificación final (tsc, lint, 37 pruebas, build, RLS 9/9)

Parte 1 — README

EcoTrack · EmissionsTracker (React + Supabase)

Proyecto Integrador «El Ciclo Spec-to-Code en Acción», módulo 3, «El Núcleo UI (React + IA)». **Entregable principal: BITACORA.md (PDF).**

> **Real y simulado.** El componente, el hook, las 37 pruebas, el build y las capturas se ejecutaron.
> Sin `.env`, la app arranca en **modo demo**: datos de ejemplo en memoria, marcados en pantalla.
> El cliente de Supabase está escrito y compilado, pero **no se ha ejecutado contra un proyecto real**,
> porque no hay credenciales. `supabase/schema.sql` se probó en PostgreSQL 17 (Docker) con un stub de Supabase: 9 de 9.

Entregable de la rúbrica → archivo

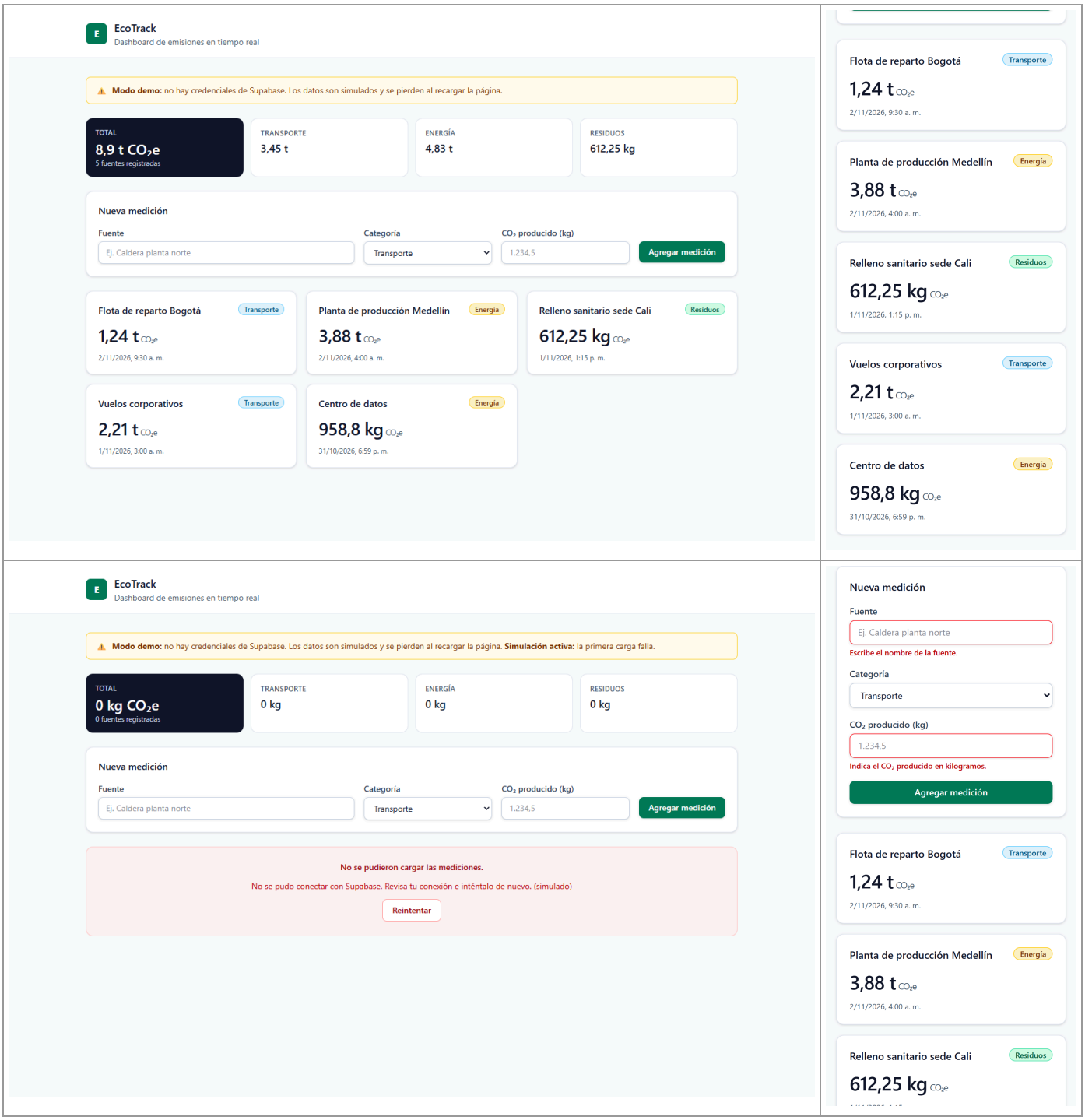
Entregable	Archivo
Bitácora de 3 o 4 prompts y la evolución del código	BITACORA.md, BITACORA.pdf y los prompts sueltos en bitacora/
Componente `EmissionsTracker.tsx`	src/components/EmissionsTracker.tsx
Hook `useEmissions.ts`	src/hooks/useEmissions.ts y sus pruebas en useEmissions.test.ts
Configuración de Supabase	src/lib/supabaseClient.ts, src/services/emissionsService.ts, .env.example y supabase/schema.sql
Capturas	docs/capturas/, con la lista en BITACORA.md §7

Criterio de la rúbrica → dónde se demuestra

Criterio	Evidencia
Alineación estratégica (prompteo iterativo)	BITACORA.md: tabla de la evolución y §1 a §4. Cada prompt cita la salida real de la versión anterior, guardada en `bitacora/vN/verificacion.txt`
Ejecución técnica (React + Supabase, TS, hooks)	Capas `components → hooks → services → lib` que oxlint hace cumplir (.oxlintrc.json); TS con `noUncheckedIndexedAccess` y `exactOptionalPropertyTypes`; verificación final
Calidad de UI y usabilidad	Tailwind responsive a 1, 2 y 3 columnas; esqueleto de carga, «Guardando…», error con «Reintentar», estado vacío y validación por campo (capturas)
Auditoría IA (código, prompts, bundle, RLS)	BITACORA.md §6

Vista rápida (capturas reales)

Escritorio	Móvil



Todas las capturas (carga, guardando, error al guardar, vacío...) están en docs/capturas/.

Árbol

```
src/
├── App.tsx                      ← encabezado y <main>
├── components/                  ← UI: solo importa hooks/, types/ y utils/
│   ├── EmissionsTracker.tsx     ← contenedor: resumen, formulario, estados y grid
│   ├── EmissionCard.tsx         ← tarjeta (memo)
│   ├── EmissionForm.tsx         ← formulario accesible con validación por campo
│   ├── EmissionsStatus.tsx      ← esqueleto, error con reintento y vacío
│   └── EmissionsSummary.tsx     ← total y desglose por categoría (memo)
├── hooks/useEmissions.ts        ← carga, inserción, errores, doble envío y modo
├── services/
│   ├── emissionsService.ts      ← Supabase: columnas, validación de filas y mensajes de error
│   ├── demoRepository.ts        ← MODO DEMO en memoria y ?simular=
│   └── index.ts                 ← elige Supabase o demo según las variables
├── lib/supabaseClient.ts        ← único createClient; null sin credenciales
├── types/emissions.ts           ← tipos de dominio y contratos
└── utils/                       ← formato, resumen y validación (funciones puras)
supabase/schema.sql              ← tabla, checks, privilegios por columna y RLS
supabase/pruebas/                ← stub de Supabase y 9 pruebas de RLS
```

```
bitacora/                          ← prompts 01 a 04 y las versiones v1 a v4 tal cual, con su verificación
herramientas/                      ← verificar-version.sh, verificacion-final.sh y probar-rls.sh
docs/capturas/, docs/evidencia/    ← capturas y salidas reales de las herramientas
```

Cómo ejecutarlo

```
pnpm install
pnpm dev                 # modo demo; prueba también ?simular=lento|error-carga|error-guardado|vacio
pnpm typecheck           # tsc -b (ojo: `tsc --noEmit` solo no revisa nada aquí, ver BITACORA §6.1)
pnpm lint
pnpm test
pnpm build
bash herramientas/probar-rls.sh          # requiere Docker
```

Para usar Supabase de verdad:

1. Copia `.env.example` a `.env` y llena `VITE_SUPABASE_URL` y `VITE_SUPABASE_ANON_KEY`. Solo la anon

key; la service_role nunca va en el frontend.

2. Ejecuta `supabase/schema.sql` en el SQL Editor del proyecto.

Parte 2 — Bitácora de prompts

Bitácora de prompts · EcoTrack EmissionsTracker

Proyecto Integrador «El Ciclo Spec-to-Code en Acción», módulo 3, «El Núcleo UI (React + IA)». Stack: Vite 8, React 19, TypeScript 6 en modo estricto, Tailwind CSS 4, Supabase JS 2 y Vitest 5, todo con pnpm.

Esta bitácora recoge los cuatro prompts clave y cómo cambió el código de uno a otro. Ninguna versión se reconstruyó al final: cada respuesta de la IA se guardó tal cual en `bitacora/vN/` y se verificó con las mismas herramientas antes de escribir el siguiente prompt. Cada prompt parte de esas salidas reales (errores de `tsc`, avisos de `oxlint`, pruebas que fallan y pruebas de RLS), no de una idea general de «mejorar el código».

Paso	Prompt	Versión	tsc	oxlint	Pruebas	RLS (anon)	Bundle JS con variables
1	Estructura	`v1`	0 errores	0	sin pruebas	sin SQL	226,15 kB (no usa Supabase)
2	Lógica del hook	`v2`	0 errores	2 avisos	**2 de 4 fallan**	**4 de 9**	444,55 kB
3	Refinamiento	`v3`	0 errores (modo más estricto)	**1 error, 2 avisos**	27 de 27	sin cambios	448,34 kB
4	Auditoría	`v4`	0 errores	**0**	37 de 37	**9 de 9**	449,13 kB
Final	Ajuste tras las capturas	`src/`	0 errores	0	37 de 37	9 de 9	449,37 kB (128,33 kB gzip)

Fuentes: `bitacora/v1/verificacion.txt` a `v4/verificacion.txt`, `v2/rls-pruebas.txt`, `v4/rls-pruebas.txt` y `docs/evidencia/verificacion-final.txt`.

---

1. Prompt de estructura

Qué se buscaba: que la IA propusiera la arquitectura a partir de la especificación, sin escribir todavía la lógica. Se le dio el stack exacto (versiones, `verbatimModuleSyntax` y Tailwind 4 sin archivo de configuración) y una regla de dependencias: la UI nunca importa Supabase.

```
Actúa como arquitecto frontend senior. Vamos a construir el núcleo de la interfaz del
"Dashboard de Emisiones en Tiempo Real" de EcoTrack […]
Stack fijo (ya instalado, no propongas otro): Vite 8 + React 19 + TypeScript 6 con "strict": true
y "verbatimModuleSyntax": true; Tailwind CSS 4 con @tailwindcss/vite; @supabase/supabase-js 2.
[…]
En ESTE paso solo quiero la arquitectura, no la lógica de datos:
1. Propón el árbol de archivos de src/ y la responsabilidad de cada archivo. Regla: la UI
   (components/) nunca importa Supabase; solo habla con el hook.
2. Escribe los tipos de dominio en src/types/emissions.ts.
3. Escribe una primera versión de EmissionsTracker.tsx con datos de ejemplo en estado local […]
5. Lista los estados de la interfaz que el hook tendrá que exponer en el siguiente paso.
```

Prompt completo: `bitacora/01-prompt-estructura.md`. Respuesta: `bitacora/v1/respuesta.md`.

Qué devolvió (v1):

- Árbol por capas (`types → services → hooks → components`).
- Tipos sin `enum`, porque `erasableSyntaxOnly` no los permite: `EMISSION_CATEGORIES` como tupla

`as const`, de la que sale el tipo.

- El contrato `UseEmissionsResult`, escrito antes que el hook.
- Un componente de 315 líneas con `useState` y cinco mediciones de ejemplo.

Qué se vio al verificar: compila y pasa el lint, pero los datos siguen en el componente, el total se recalcula en cada tecla, `replace(',', '.')` cambia solo la primera coma (así que `1.234,5` da `NaN`) y el error de categoría se calcula pero nunca se muestra.

---

2. Prompt de lógica: el hook useEmissions

Qué se buscaba: sacar los datos de la UI. Se pidieron el cliente de Supabase que no falla sin credenciales (modo demo), un servicio que valide cada fila en vez de usar `as`, inserción pesimista, errores de carga e inserción separados, protección contra el doble montaje de StrictMode, el repositorio inyectable para probar sin red, `schema.sql` con RLS y cuatro pruebas.

```
1. src/lib/supabaseClient.ts: crea el cliente SOLO si existen VITE_SUPABASE_URL y
   VITE_SUPABASE_ANON_KEY. Si falta alguna, exporta null en vez de lanzar un error […]
2. src/services/emissionsService.ts: […] valida cada fila que llega: si la categoría no es
   una de las tres, descarta la fila en vez de hacer un cast con "as".
```

```
   Ojo: numeric de Postgres puede llegar como string por PostgREST.
4. src/hooks/useEmissions.ts que implemente UseEmissionsResult: […] El hook recibe el
   repositorio por parámetro (con el real o el demo por defecto) para poder probarlo con
   Vitest sin red.
6. supabase/schema.sql con la tabla, un check de categoría y de co2_kg >= 0, y RLS.
7. Pruebas de Vitest para el hook con un repositorio falso […]
```

Prompt completo: `bitacora/02-prompt-logica-hook.md`. Diferencias con v1: `bitacora/v2/cambios-desde-v1.diff` (10 archivos, +580 −277).

El cambio central, en `EmissionsTracker.tsx`:

```
-  // Paso 1: estado local. Paso 2: const { emissions, ... } = useEmissions()
-  const [emissions, setEmissions] = useState<readonly Emission[]>(SAMPLE_EMISSIONS)
+  const { mode, emissions, status, loadError, isSubmitting, submitError, addEmission, reload, clearSubmitError } =
+    useEmissions()
```

Qué se vio al verificar (`v2/verificacion.txt`):

- **2 de 4 pruebas fallan por timeout.** La prueba crea el repositorio dentro de

`renderHook(() => useEmissions(fakeRepository()))`, así que en cada render llega uno nuevo; como `reload` depende de él, el efecto vuelve a cargar sin fin. La prueba no está mal: muestra un hook frágil.

- oxlint marca dos avisos: `set-state-in-effect` y `only-export-components`.
- **RLS, probado como anon en PostgreSQL 17 (`v2/rls-pruebas.txt`): 5 de 9 pruebas fallan.** El cliente

puede fijar `measured_at` y elegir el `id`, `update` y `delete` no dan error, y **`truncate` vació la tabla**. RLS habilitado con dos políticas no basta: Supabase da todos los privilegios a anon en cada tabla nueva, y RLS no filtra `truncate`.

- Bundle: 228,41 kB sin variables y 444,55 kB con variables de prueba. La diferencia es Supabase, que Vite

elimina como código muerto si no hay credenciales.

---

3. Prompt de refinamiento: TypeScript estricto, accesibilidad y useMemo

Qué se buscaba: las tres restricciones del enunciado (accesibilidad, TypeScript estricto y `useMemo`), más la corrección de cada fallo visto en v2. Cada exigencia es verificable:

- banderas concretas del compilador;
- «ningún `as` salvo `as const`»;
- una lista de entradas que el parser debe aceptar y rechazar;
- roles ARIA concretos;
- `memo` y `useMemo` en lugares concretos.

```
A. TypeScript estricto de verdad
   - Activa "noUncheckedIndexedAccess", "exactOptionalPropertyTypes" y "noImplicitOverride" […]
   - Ningún "as" salvo "as const". parseEmissionRow tiene que estrechar el tipo con
     comprobaciones, no con "row as Record<string, unknown>".
B. Bugs encontrados en la verificación
   - 2 pruebas fallan por timeout: el hook recarga sin fin cuando el repositorio no es estable
     entre renders. El hook debe ser robusto a eso sin pedirle nada a quien lo llama. […]
   - Debe aceptar "1.234,5" (formato colombiano), "1234.5" y "1234,5"; rechazar "1,2,3", "abc",
     "-3", "Infinity" y el vacío.
   - Evita doble envío […]. Agrega la prueba.
C. Accesibilidad (WCAG 2.1 AA): role="status"/aria-live, role="alert", foco al primer campo
   con error, error de categoría visible, total legible para lectores de pantalla […]
D. Re-renders: useMemo para el total y el desglose; React.memo en EmissionCard; el objeto
   que devuelve el hook memorizado con useMemo […]
```

Prompt completo: `bitacora/03-prompt-refinamiento.md`. Diferencias con v2: `bitacora/v3/cambios-desde-v2.diff` (13 archivos, +541 −239).

La corrección del bug de v2, en `useEmissions.ts`:

```
 export function useEmissions(repository: EmissionsRepository = defaultRepository): UseEmissionsResult {
+  const [repo] = useState(() => repository)
+
   const [emissions, setEmissions] = useState<readonly Emission[]>([])
-  const [status, setStatus] = useState<LoadStatus>('idle')
+  // Empieza en 'loading': así el efecto no necesita un setState síncrono.
+  const [status, setStatus] = useState<LoadStatus>('loading')
```

Errores atrapados por `exactOptionalPropertyTypes`: v2 borraba un error con `setErrors((prev) => ({ ...prev, [field]: undefined }))`. Con la bandera nueva eso ya no compila, y v3 quita la clave con `withoutError()`. Lo mismo pasa con `aria-describedby=`

{cond ? id : undefined}, que v3 resuelve con props condicionales.

Re-renders: el resumen pasa a useMemo(() => summarizeEmissions(emissions), [emissions]), y EmissionCard y EmissionsSummary se envuelven en memo. Hay una prueba que comprueba que el objeto que devuelve el hook es el mismo tras un rerender sin cambios.

Qué se vio al verificar (v3/verificacion.txt): tsc sin errores con las banderas nuevas y **27 de 27 pruebas verdes**: 7 del hook, entre ellas la regresión del repositorio inestable y el doble envío, y 20 de validación. Pero **oxlint pasa de 2 avisos a 1 error y 2 avisos**:

- exhaustive-deps: se lee un ref en el cleanup.
- set-state-in-effect: sigue ahí. Separar fetchList de reload no bastó.
- prefer-tag-over-role: aparece por el role="status" nuevo.

El refinamiento arregló lo que se pidió y dejó a la vista lo que el linter todavía no aceptaba.

---

4. Prompt de auditoría: lint en cero, RLS y estados verificables

Qué se buscaba: cerrar los cuatro ejes de la Auditoría IA del módulo. Se le dio a la IA la salida exacta de oxlint y de las pruebas de RLS, con instrucciones de arquitectura concretas: la carga como suscripción con bandera local, <output> en vez de role="status", privilegios por columna y un ?simular= que solo actúa en modo demo.

```
1. Lint en cero (errores y avisos):
   - No leas ref.current en el cleanup del efecto.
   - Ningún setState alcanzable de forma síncrona desde el efecto de carga. Escribe la carga
     inicial como una suscripción: el efecto lanza la promesa, guarda una bandera local
     "activo" y solo aplica el resultado en el .then si sigue activo. […]
   - Haz cumplir la arquitectura en .oxlintrc.json con no-restricted-imports […]
2. RLS: revoke all a anon y authenticated; grant select, y grant insert SOLO sobre las
   columnas source_name, category y co2_kg. Sin update, delete ni truncate. […]
   Debe pasar las 9 pruebas de supabase/pruebas/rls.sql.
3. Estados verificables en modo demo: ?simular=lento | error-carga | error-guardado | vacio […]
```

Prompt completo: bitacora/04-prompt-auditoria.md. Diferencias con v3: bitacora/v4/cambios-desde-v3.diff (9 archivos, +153 −49).

La carga como suscripción (useEmissions.ts):

```
useEffect(() => {
  let active = true
  repo.list().then(
    (rows) => { if (!active) return; setEmissions(rows); setStatus('success') },
    (error: unknown) => { if (!active) return; setLoadError(messageOf(error)); setStatus('error') },
  )
  return () => { active = false }
}, [repo, loadRequest])
```

Privilegios antes que políticas (supabase/schema.sql):

```
revoke all on table public.emissions from public, anon, authenticated;
grant select on table public.emissions to anon, authenticated;
grant insert (source_name, category, co2_kg) on table public.emissions to anon, authenticated;
```

Qué se vio al verificar: tsc sin errores, **oxlint en 0**, **37 de 37 pruebas** (10 nuevas del modo demo) y **RLS 9 de 9**. El schema se aplicó dos veces sin error; el de v2 fallaba la segunda vez. La regla de arquitectura se probó con archivos que tenían imports prohibidos, y oxlint dio los 4 errores esperados (v4/lint-regla-arquitectura.txt).

---

5. Ajuste final tras revisar las capturas

La captura de carga lenta mostró algo que ninguna herramienta había detectado: mientras carga, el resumen decía **«0 kg CO_2e»**, como si la empresa no emitiera nada. Se cambió a mano para que muestre «—» con aria-busy mientras carga, y se agregó motion-reduce:animate-none al esqueleto y al spinner. Diferencias: bitacora/ajuste-final-tras-capturas.diff. Esta es la versión que queda en src/.

---

6. Auditoría IA

6.1 Análisis de código

Verificación	Resultado	Evidencia
pnpm typecheck (tsc -b --noEmit) con strict, noUncheckedIndexedAccess y exactOptionalPropertyTypes	0 errores	docs/evidencia/verificacion-final.txt
pnpm lint (oxlint con react, typescript y jsx-a11y)	0 errores, 0 avisos	ídem

La regla de arquitectura (`no-restricted-imports`) detecta imports prohibidos	4 de 4 detectados	`bitacora/v4/lint-regla-arquitectura.txt`
`pnpm test` (Vitest, happy-dom)	37 de 37	ídem
Casts con `as` en `src/`, sin contar `as const` ni las pruebas	0 (el único `as` que aparece es un alias de import)	`grep` sobre `src/`

Hallazgo de la auditoría: **`pnpm tsc --noEmit` no revisa nada en este proyecto**. El `tsconfig.json` raíz tiene `"files": []` y solo referencias, así que el comando sale con 0 incluso con un error de tipos a propósito. `tsc -b` sí lo detecta (`docs/evidencia/tsc-noemit-no-revisa-nada.txt`). Por eso el chequeo real es el script `typecheck`.

6.2 Optimización de prompts

Prompt	Lo que lo volvió específico
1	Versiones exactas y opciones del compilador; prohibido proponer otro stack; solo arquitectura
2	Contrato del paso 1 como entrada; cada requisito del hook se puede comprobar (inserción pesimista, errores separados, StrictMode, inyección)
3	Salida real de las herramientas como contexto; casos de entrada concretos; banderas concretas del compilador
4	Errores exactos de oxlint y de RLS; la solución de arquitectura indicada en una frase; criterio de aceptación numérico (9 de 9, lint en 0)

Lo que no funcionó: en el prompt 3, separar `fetchList` de `reload` no eliminó `set-state-in-effect`. Pedir «quita el aviso» no bastó: hubo que describir el patrón (una suscripción con bandera local) en el prompt 4.

6.3 Tamaño del bundle

Compilación	JS	JS gzip	CSS	CSS gzip
Sin variables de entorno (modo demo)	233,20 kB	73,15 kB	18,53 kB	4,63 kB
Con variables de prueba (valores falsos)	**449,37 kB**	**128,33 kB**	18,53 kB	4,63 kB

El segundo es el tamaño que verá el usuario con Supabase. Sin credenciales, Vite reemplaza `import.meta.env.VITE_*` por `undefined` y elimina `@supabase/supabase-js` como código muerto. Si solo se midiera el primero, el tamaño quedaría subestimado en unos 216 kB.

6.4 Verificación RLS

Prueba: `bash herramientas/probar-rls.sh`. Crea un PostgreSQL 17 desechable en Docker con un stub de los roles y privilegios por defecto de Supabase, aplica el schema dos veces y corre 9 pruebas como `anon`.

#	Prueba	v2	v4
1	anon lee	OK	OK
2	anon inserta una medición válida	OK	OK
3	Categoría fuera de la lista	OK (check)	OK (política)
4	`co2_kg` negativo	OK (check)	OK (política)
5	El cliente fija `measured_at`	**FALLA**	OK
6	El cliente elige el `id`	**FALLA**	OK
7	`update`	**FALLA** (sin error)	OK (denegado)
8	`delete`	**FALLA** (sin error)	OK (denegado)
9	`truncate`	**FALLA** (vació la tabla)	OK (denegado)

Límite: el stub reproduce los roles y privilegios por defecto de Supabase, pero no es Supabase. `schema.sql` no se ha ejecutado en un proyecto real porque no hay credenciales.

---

7. Capturas

Todas están en `docs/capturas/`, en modo demo, servidas con `vite preview` y sacadas en Chrome a 1366×900 (escritorio) y 390×844 (móvil).

Archivo	Qué muestra
`01-escritorio-lista.png`	Resumen por categoría y las cinco cards
`02-escritorio-validacion.png`	Errores por campo y foco en el primero

`03-escritorio-guardando.png`	Botón «Guardando…» deshabilitado
`04-escritorio-medicion-agregada.png`	«Caldera planta norte» (1.234,5 kg) primera en la lista; total en 10,13 t
`05-carga-esqueleto.png`	`?simular=lento`: esqueleto y resumen en «—»
`06-error-de-carga.png`	`?simular=error-carga`: mensaje y «Reintentar»
`07-lista-vacia.png`	`?simular=vacio`
`08-error-al-guardar.png`	`?simular=error-guardado`: mensaje de RLS y lo escrito se conserva
`09-movil-inicio.png`, `10-movil-cards.png`, `11-movil-validacion.png`	Móvil a una columna, sin scroll horizontal (`scrollWidth` 390 = `innerWidth` 390)

---

8. Qué es real y qué es simulado

Pieza	Estado
Componente, hook, validación y pruebas	Ejecutado: 37 pruebas, build y capturas
Modo demo	Real y marcado en pantalla. Datos de ejemplo, no mediciones de una empresa
Cliente y servicio de Supabase	Compilados y con los tipos revisados. **No ejecutados contra un proyecto real** (no hay credenciales)
`schema.sql` y RLS	Ejecutado en PostgreSQL 17 con un stub de Supabase. No ejecutado en Supabase
Las cifras de CO_2	De ejemplo. No salen de factores de emisión

Parte 3 — Código fuente

src/components/EmissionsTracker.tsx

```
import { useMemo } from 'react'
import { useEmissions } from '../hooks/useEmissions.ts'
import type { DemoSimulation } from '../types/emissions.ts'
import { summarizeEmissions } from '../utils/format.ts'
import { EmissionCard } from './EmissionCard.tsx'
import { EmissionForm } from './EmissionForm.tsx'
import { EmissionsEmpty, EmissionsLoadError, EmissionsLoading } from './EmissionsStatus.tsx'
import { EmissionsSummary } from './EmissionsSummary.tsx'

const SIMULATION_LABELS: Record<DemoSimulation, string> = {
  lento: 'carga lenta (4 s)',
  'error-carga': 'la primera carga falla',
  'error-guardado': 'todo guardado falla',
  vacio: 'sin mediciones',
}

/*
 * Contenedor del dashboard. Toda la lógica de datos está en useEmissions():
 * este archivo no importa Supabase, ni lib/, ni services/ (lo vigila .oxlintrc.json).
 */
export default function EmissionsTracker() {
  const { mode, simulation, emissions, status, loadError, isSubmitting, submitError, addEmission, reload, clearSubmitError } =
    useEmissions()

  // Solo se recalcula cuando cambia la lista, no en cada tecla del formulario.
  const summary = useMemo(() => summarizeEmissions(emissions), [emissions])

  return (
    <section aria-labelledby="emissions-title" className="flex flex-col gap-6">
      {mode === 'demo' && (
        <p className="flex items-start gap-2 rounded-lg border border-amber-300 bg-amber-50 px-4 py-3 text-sm text-amber-900">
          <span aria-hidden="true">⚠</span>
          <span>
            <strong>Modo demo:</strong> no hay credenciales de Supabase. Los datos son simulados y se pierden al
            recargar la página.
            {simulation && (
              <>
                {' '}
                <strong>Simulación activa:</strong> {SIMULATION_LABELS[simulation]}.
              </>
            )}
          </span>
        </p>
      )}

      <h2 id="emissions-title" className="sr-only">
        Mediciones de emisiones
      </h2>

      <EmissionsSummary summary={summary} loading={status === 'loading'} />

      <EmissionForm onAdd={addEmission} isSubmitting={isSubmitting} submitError={submitError} onEdit={clearSubmitError} />

      {status === 'loading' && <EmissionsLoading />}

      {status === 'error' && <EmissionsLoadError message={loadError ?? 'Error desconocido.'} onRetry={reload} />}

      {status === 'success' && emissions.length === 0 && <EmissionsEmpty />}

      {status === 'success' && emissions.length > 0 && (
        <ul aria-label="Mediciones registradas" className="grid gap-4 sm:grid-cols-2 lg:grid-cols-3">
          {emissions.map((emission) => (
            <li key={emission.id}>
              <EmissionCard emission={emission} />
            </li>
          ))}
        </ul>
      )}
    </section>
  )
}
```

src/hooks/useEmissions.ts

```
import { useCallback, useEffect, useMemo, useRef, useState } from 'react'
import { defaultRepository } from '../services/index.ts'
import type {
  Emission,
  EmissionsRepository,
  LoadStatus,
  NewEmission,
  UseEmissionsResult,
} from '../types/emissions.ts'

function messageOf(error: unknown): string {
  return error instanceof Error ? error.message : 'Ocurrió un error inesperado.'
```

```
}

/**
 * Estado de la pantalla de emisiones: carga, inserción y errores.
 *
 * - El repositorio entra por parámetro (Supabase o demo por defecto) para probar
 *   sin red. Se fija el del PRIMER render: si quien llama crea uno nuevo en cada
 *   render, el hook no recarga sin fin (bug encontrado en v2).
 * - La carga es una suscripción: el efecto lanza la promesa y solo aplica el
 *   resultado si sigue activo (StrictMode monta dos veces; "Reintentar" puede
 *   llegar antes que una respuesta lenta).
 * - La UI solo habla con este hook; nunca con services/ ni con Supabase.
 */
export function useEmissions(repository: EmissionsRepository = defaultRepository): UseEmissionsResult {
  const [repo] = useState(() => repository)

  const [emissions, setEmissions] = useState<readonly Emission[]>([])
  const [status, setStatus] = useState<LoadStatus>('loading')
  const [loadError, setLoadError] = useState<string | null>(null)
  const [isSubmitting, setIsSubmitting] = useState(false)
  const [submitError, setSubmitError] = useState<string | null>(null)
  // Cambiarlo vuelve a disparar el efecto de carga ("Reintentar").
  const [loadRequest, setLoadRequest] = useState(0)

  const mountedRef = useRef(false)
  // Bloqueo síncrono de doble envío: el estado isSubmitting llega un render tarde.
  const submittingRef = useRef(false)

  useEffect(() => {
    mountedRef.current = true
    return () => {
      mountedRef.current = false
    }
  }, [])

  useEffect(() => {
    let active = true
    repo.list().then(
      (rows) => {
        if (!active) return
        setEmissions(rows)
        setStatus('success')
      },
      (error: unknown) => {
        if (!active) return
        setLoadError(messageOf(error))
        setStatus('error')
      },
    )
    return () => {
      active = false
    }
  }, [repo, loadRequest])

  const reload = useCallback(() => {
    setStatus('loading')
    setLoadError(null)
    setLoadRequest((n) => n + 1)
  }, [])

  const addEmission = useCallback(
    async (input: NewEmission) => {
      if (submittingRef.current) return false
      submittingRef.current = true
      setIsSubmitting(true)
      setSubmitError(null)
      try {
        // Pesimista: el id y la fecha vienen de la base de datos.
        const created = await repo.insert(input)
        if (mountedRef.current) setEmissions((prev) => [created, ...prev])
        return true
      } catch (error) {
        if (mountedRef.current) setSubmitError(messageOf(error))
        return false
      } finally {
        submittingRef.current = false
        if (mountedRef.current) setIsSubmitting(false)
      }
    },
    [repo],
  )

  const clearSubmitError = useCallback(() => setSubmitError(null), [])

  // Objeto estable mientras nada cambie: los componentes memorizados no se re-renderizan de más.
  return useMemo(
    () => ({
      mode: repo.mode,
      simulation: repo.simulation,
      emissions,
      status,
      loadError,
      isSubmitting,
      submitError,
      addEmission,
      reload,
      clearSubmitError,
    }),
```

```
    [repo, emissions, status, loadError, isSubmitting, submitError, addEmission, reload, clearSubmitError],
  )
}
```

src/lib/supabaseClient.ts

```
import { createClient } from '@supabase/supabase-js'
import type { SupabaseClient } from '@supabase/supabase-js'

/*
 * Único lugar que crea el cliente de Supabase. Solo lo importa services/.
 *
 * Las credenciales llegan por variables de entorno de Vite (archivo .env, que no
 * se sube al repositorio; ver .env.example). Solo se usa la anon key: la
 * service_role nunca va en el frontend.
 *
 * Si falta alguna variable no se lanza un error: se exporta null y la app
 * arranca en "modo demo" con datos simulados.
 */
const url = import.meta.env.VITE_SUPABASE_URL
const anonKey = import.meta.env.VITE_SUPABASE_ANON_KEY

export const supabase: SupabaseClient | null = url && anonKey ? createClient(url, anonKey) : null
```

src/services/emissionsService.ts

```
import type { PostgrestError, SupabaseClient } from '@supabase/supabase-js'
import { isEmissionCategory } from '../types/emissions.ts'
import type { Emission, EmissionsRepository, NewEmission } from '../types/emissions.ts'

/*
 * Acceso a la tabla public.emissions. Es el único archivo que conoce los nombres
 * de las columnas (snake_case); hacia arriba solo salen tipos de dominio.
 */

const TABLE = 'emissions'
const COLUMNS = 'id, source_name, category, co2_kg, measured_at'

function isRecord(value: unknown): value is Record<string, unknown> {
  return typeof value === 'object' && value !== null
}

/** numeric de Postgres puede llegar por PostgREST como number o como string. */
function toNumber(value: unknown): number | null {
  const n = typeof value === 'string' && value.trim() !== '' ? Number(value) : value
  return typeof n === 'number' && Number.isFinite(n) ? n : null
}

/** Convierte una fila cruda en Emission, o null si no es válida. Estrecha tipos sin casts. */
export function parseEmissionRow(row: unknown): Emission | null {
  if (!isRecord(row)) return null
  const { id, source_name, category, measured_at } = row
  const co2Kg = toNumber(row.co2_kg)

  if (typeof id !== 'string' || typeof source_name !== 'string' || typeof measured_at !== 'string') return null
  if (!isEmissionCategory(category) || co2Kg === null || co2Kg < 0) return null

  return { id, sourceName: source_name, category, co2Kg, measuredAt: measured_at }
}

/** Traduce un error de Supabase a un mensaje para el usuario. */
export function toUserMessage(error: Pick<PostgrestError, 'message' | 'code'>): string {
  if (error.code === '42501') return 'No tienes permiso para esta operación (política RLS).'
  if (error.code === '23514') return 'Los datos no cumplen las reglas de la tabla.'
  if (error.message.includes('Failed to fetch')) {
    return 'No se pudo conectar con Supabase. Revisa tu conexión e inténtalo de nuevo.'
  }
  return `Error de Supabase: ${error.message}`
}

export function createSupabaseRepository(client: SupabaseClient): EmissionsRepository {
  return {
    mode: 'supabase',
    simulation: null,

    async list() {
      const { data, error } = await client
        .from(TABLE)
        .select(COLUMNS)
        .order('measured_at', { ascending: false })
      if (error) throw new Error(toUserMessage(error))

      const rows: readonly unknown[] = data ?? []
      // Una fila inválida se descarta en vez de romper toda la lista.
      return rows.map(parseEmissionRow).filter((e): e is Emission => e !== null)
    },

    async insert(input: NewEmission) {
      // Solo las columnas que el usuario aporta: id y measured_at los pone la BD.
      const { data, error } = await client
        .from(TABLE)
        .insert({ source_name: input.sourceName, category: input.category, co2_kg: input.co2Kg })
        .select(COLUMNS)
```

```
        .single()
      if (error) throw new Error(toUserMessage(error))

      const created = parseEmissionRow(data)
      if (!created) throw new Error('La base de datos devolvió una fila con un formato inesperado.')
      return created
    },
  }
}
```

src/services/index.ts

```
import { supabase } from '../lib/supabaseClient.ts'
import type { EmissionsRepository } from '../types/emissions.ts'
import { createDemoRepository, simulationFromSearch } from './demoRepository.ts'
import { createSupabaseRepository } from './emissionsService.ts'

/**
 * Repositorio por defecto: Supabase si hay credenciales; si no, el modo demo.
 * ?simular=… solo se lee en modo demo: con Supabase no tiene ningún efecto.
 */
export const defaultRepository: EmissionsRepository = supabase
  ? createSupabaseRepository(supabase)
  : createDemoRepository({ simulation: simulationFromSearch(window.location.search) })
```

src/types/emissions.ts

```
/**
 * Tipos de dominio del Dashboard de Emisiones de EcoTrack.
 *
 * Este archivo no importa nada: ni React ni Supabase. Lo usan por igual la UI
 * (components/), el hook (hooks/) y la capa de datos (services/), y es el único
 * contrato que comparten.
 *
 * Con "erasableSyntaxOnly" no se pueden usar `enum`: las categorías se definen
 * como una tupla `as const` y el tipo se deriva de ella, así la lista para el
 * <select> y el tipo nunca se desincronizan.
 */

/** Categorías admitidas. El orden es el que se muestra en el formulario. */
export const EMISSION_CATEGORIES = ['Transporte', 'Energía', 'Residuos'] as const

/** 'Transporte' | 'Energía' | 'Residuos' */
export type EmissionCategory = (typeof EMISSION_CATEGORIES)[number]

/** Guarda de tipo para valores que llegan sin tipo (formulario, BD). Sin casts. */
export function isEmissionCategory(value: unknown): value is EmissionCategory {
  return EMISSION_CATEGORIES.some((category) => category === value)
}

/** Una medición tal como la consume la interfaz. */
export interface Emission {
  /** Identificador único (uuid generado por la BD; en modo demo, `demo-…`). */
  readonly id: string
  /** Nombre de la fuente emisora, p. ej. "Flota de reparto Bogotá". */
  readonly sourceName: string
  readonly category: EmissionCategory
  /** CO2 equivalente producido, en kilogramos. Siempre >= 0. */
  readonly co2Kg: number
  /** Fecha y hora de la medición en ISO 8601 (UTC). La asigna la BD. */
  readonly measuredAt: string
}

/** Datos que el usuario aporta para crear una medición. El id y la fecha los asigna la BD. */
export type NewEmission = Pick<Emission, 'sourceName' | 'category' | 'co2Kg'>

/**
 * Valores crudos del formulario: todo es string mientras el usuario escribe.
 * Se convierten a NewEmission solo después de validar.
 */
export interface EmissionFormValues {
  sourceName: string
  category: EmissionCategory
  co2Kg: string
}

export type EmissionFormField = keyof EmissionFormValues

/**
 * Errores de validación por campo. Con exactOptionalPropertyTypes un campo sin
 * error se OMITE: `{ co2Kg: undefined }` no compila.
 */
export type EmissionFormErrors = { readonly [K in EmissionFormField]?: string }

/** Estado de la carga del listado. Empieza en 'loading': no hay un "idle" que pintar. */
export type LoadStatus = 'loading' | 'success' | 'error'

/** De dónde salen los datos: Supabase real o el repositorio en memoria (modo demo). */
export type DataMode = 'supabase' | 'demo'

/** Casos que el modo demo puede reproducir con ?simular=… (solo para revisar la UI). */
export const DEMO_SIMULATIONS = ['lento', 'error-carga', 'error-guardado', 'vacio'] as const
export type DemoSimulation = (typeof DEMO_SIMULATIONS)[number]
```

```
/**
 * Lo que el hook necesita de la capa de datos. Lo implementan el servicio de
 * Supabase y el repositorio demo, y en las pruebas un repositorio falso.
 * Los métodos lanzan Error con un mensaje ya legible para el usuario.
 */
export interface EmissionsRepository {
  readonly mode: DataMode
  /** Simulación activa en modo demo; siempre null con Supabase. */
  readonly simulation: DemoSimulation | null
  list(): Promise<Emission[]>
  insert(input: NewEmission): Promise<Emission>
}

/** Total y desglose por categoría, derivados de la lista. */
export interface EmissionsSummary {
  readonly totalKg: number
  readonly count: number
  readonly byCategory: Readonly<Record<EmissionCategory, number>>
}

/** Contrato del hook useEmissions. La UI se escribe contra él y no contra Supabase. */
export interface UseEmissionsResult {
  /** 'demo' cuando faltan las variables de Supabase; la UI lo avisa. */
  readonly mode: DataMode
  /** Simulación activa en modo demo (?simular=…); null si no hay. */
  readonly simulation: DemoSimulation | null
  /** Mediciones ya cargadas, de la más reciente a la más antigua. */
  readonly emissions: readonly Emission[]
  readonly status: LoadStatus
  /** Mensaje legible si la carga falló; null en otro caso. */
  readonly loadError: string | null
  /** true mientras hay una inserción en curso (deshabilita el botón). */
  readonly isSubmitting: boolean
  /** Mensaje legible si la última inserción falló; null en otro caso. */
  readonly submitError: string | null
  /** Inserta una medición. true si se guardó; false si falló o ya había otra en curso. */
  readonly addEmission: (input: NewEmission) => Promise<boolean>
  /** Vuelve a pedir el listado (botón "Reintentar"). El resultado llega por status. */
  readonly reload: () => void
  /** Limpia submitError cuando el usuario vuelve a editar el formulario. */
  readonly clearSubmitError: () => void
}
```

src/components/EmissionForm.tsx

```
import { useId, useRef, useState } from 'react'
import type { FormEvent } from 'react'
import { EMISSION_CATEGORIES, isEmissionCategory } from '../types/emissions.ts'
import type { EmissionFormErrors, EmissionFormField, EmissionFormValues, NewEmission } from '../types/emissions.ts'
import { MAX_SOURCE_NAME, validateEmissionForm, withoutError } from '../utils/validation.ts'

const EMPTY_FORM: EmissionFormValues = { sourceName: '', category: 'Transporte', co2Kg: '' }

/** Orden visual de los campos: el foco va al primero que tenga error. */
const FIELD_ORDER: readonly EmissionFormField[] = ['sourceName', 'category', 'co2Kg']

interface EmissionFormProps {
  readonly onAdd: (input: NewEmission) => Promise<boolean>
  readonly isSubmitting: boolean
  readonly submitError: string | null
  readonly onEdit: () => void
}

const inputBase =
  'w-full rounded-lg border bg-white px-3 py-2 text-sm text-slate-900 shadow-sm transition focus:outline-none focus-visible:ring-2 focus-
visible:ring-emerald-600'
const inputClass = (hasError: boolean) => `${inputBase} ${hasError ? 'border-red-500' : 'border-slate-300'}`

export function EmissionForm({ onAdd, isSubmitting, submitError, onEdit }: EmissionFormProps) {
  const [values, setValues] = useState<EmissionFormValues>(EMPTY_FORM)
  const [errors, setErrors] = useState<EmissionFormErrors>({})
  const idPrefix = useId()
  const fieldRefs = useRef<Partial<Record<EmissionFormField, HTMLInputElement | HTMLSelectElement | null>>>({})

  const ids = {
    sourceName: `${idPrefix}-source`,
    category: `${idPrefix}-category`,
    co2Kg: `${idPrefix}-co2`,
  } satisfies Record<EmissionFormField, string>

  function update<K extends EmissionFormField>(field: K, value: EmissionFormValues[K]) {
    setValues((prev) => ({ ...prev, [field]: value }))
    setErrors((prev) => withoutError(prev, field))
    if (submitError) onEdit()
  }

  async function handleSubmit(event: FormEvent<HTMLFormElement>) {
    event.preventDefault()
    const result = validateEmissionForm(values)
    if (!result.ok) {
      setErrors(result.errors)
      const firstInvalid = FIELD_ORDER.find((field) => result.errors[field] !== undefined)
      if (firstInvalid) fieldRefs.current[firstInvalid]?.focus()
      return
```

```
    }
    // Solo se limpia si se guardó; si falla, el usuario no pierde lo que escribió.
    const saved = await onAdd(result.value)
    if (saved) {
      setValues(EMPTY_FORM)
      setErrors({})
    }
  }

  /** Props ARIA de un campo: aria-describedby solo si hay error (exactOptionalPropertyTypes). */
  function a11y(field: EmissionFormField) {
    const error = errors[field]
    return error ? { 'aria-invalid': true, 'aria-describedby': `${ids[field]}-error` } : { 'aria-invalid': false }
  }

  function fieldError(field: EmissionFormField) {
    const error = errors[field]
    return error ? (
      <p id={`${ids[field]}-error`} className="text-xs font-medium text-red-700">
        {error}
      </p>
    ) : null
  }

  return (
    <form
      onSubmit={handleSubmit}
      noValidate
      aria-labelledby={`${idPrefix}-title`}
      className="grid gap-4 rounded-xl border border-slate-200 bg-white p-5 shadow-sm sm:grid-cols-2 lg:grid-cols-[2fr_1fr_1fr_auto]
lg:items-start"
    >
      <h2 id={`${idPrefix}-title`} className="text-base font-semibold text-slate-900 sm:col-span-2 lg:col-span-4">
        Nueva medición
      </h2>

      <div className="flex flex-col gap-1 sm:col-span-2 lg:col-span-1">
        <label htmlFor={ids.sourceName} className="text-sm font-medium text-slate-700">
          Fuente
        </label>
        <input
          ref={(el) => {
            fieldRefs.current.sourceName = el
          }}
          id={ids.sourceName}
          type="text"
          autoComplete="off"
          maxLength={MAX_SOURCE_NAME + 20}
          value={values.sourceName}
          onChange={(e) => update('sourceName', e.target.value)}
          placeholder="Ej. Caldera planta norte"
          className={inputClass(errors.sourceName !== undefined)}
          {...a11y('sourceName')}
        />
        {fieldError('sourceName')}
      </div>

      <div className="flex flex-col gap-1">
        <label htmlFor={ids.category} className="text-sm font-medium text-slate-700">
          Categoría
        </label>
        <select
          ref={(el) => {
            fieldRefs.current.category = el
          }}
          id={ids.category}
          value={values.category}
          onChange={(e) => {
            const next = e.target.value
            if (isEmissionCategory(next)) update('category', next)
          }}
          className={inputClass(errors.category !== undefined)}
          {...a11y('category')}
        >
          {EMISSION_CATEGORIES.map((category) => (
            <option key={category} value={category}>
              {category}
            </option>
          ))}
        </select>
        {fieldError('category')}
      </div>

      <div className="flex flex-col gap-1">
        <label htmlFor={ids.co2Kg} className="text-sm font-medium text-slate-700">
          CO₂ producido (kg)
        </label>
        <input
          ref={(el) => {
            fieldRefs.current.co2Kg = el
          }}
          id={ids.co2Kg}
          type="text"
          inputMode="decimal"
          autoComplete="off"
          value={values.co2Kg}
          onChange={(e) => update('co2Kg', e.target.value)}
          placeholder="1.234,5"
```

```
          className={inputClass(errors.co2Kg !== undefined)}
          {...a11y('co2Kg')}
        />
        {fieldError('co2Kg')}
      </div>

      <button
        type="submit"
        disabled={isSubmitting}
        aria-disabled={isSubmitting}
        className="inline-flex items-center justify-center gap-2 rounded-lg bg-emerald-700 px-4 py-2 text-sm font-medium text-white
shadow-sm transition hover:bg-emerald-800 focus-visible:outline-2 focus-visible:outline-offset-2 focus-visible:outline-emerald-700
disabled:cursor-not-allowed disabled:opacity-70 sm:col-span-2 lg:col-span-1 lg:mt-6"
      >
        {isSubmitting && (
          <span aria-hidden="true" className="size-4 animate-spin rounded-full motion-reduce:animate-none border-2 border-white border-t-
transparent" />
        )}
        {isSubmitting ? 'Guardando…' : 'Agregar medición'}
      </button>

      {submitError && (
        <p role="alert" className="rounded-lg border border-red-200 bg-red-50 px-3 py-2 text-sm text-red-800 sm:col-span-2 lg:col-span-
4">
          {submitError}
        </p>
      )}
    </form>
  )
}
```

src/components/EmissionCard.tsx

```
import { memo } from 'react'
import type { Emission, EmissionCategory } from '../types/emissions.ts'
import { formatCo2, formatCo2Spoken, formatDate } from '../utils/format.ts'

/** Badges con texto -800 sobre fondo -100: contraste mayor a 4,5:1 (WCAG AA). */
const CATEGORY_STYLES: Record<EmissionCategory, string> = {
  Transporte: 'bg-sky-100 text-sky-800 ring-sky-300',
  Energía: 'bg-amber-100 text-amber-900 ring-amber-300',
  Residuos: 'bg-emerald-100 text-emerald-800 ring-emerald-300',
}

interface EmissionCardProps {
  readonly emission: Emission
}

/**
 * Tarjeta de una medición. memo: escribir en el formulario re-renderiza el
 * contenedor, pero las cards solo cambian si cambia su `emission`.
 */
export const EmissionCard = memo(function EmissionCard({ emission }: EmissionCardProps) {
  return (
    <article className="flex h-full flex-col gap-3 rounded-xl border border-slate-200 bg-white p-5 shadow-sm transition hover:shadow-md">
      <header className="flex items-start justify-between gap-3">
        <h3 className="font-medium break-words text-slate-900">{emission.sourceName}</h3>
        <span
          className={`shrink-0 rounded-full px-2.5 py-0.5 text-xs font-medium ring-1 ring-inset ${CATEGORY_STYLES[emission.category]}`}
        >
          <span className="sr-only">Categoría: </span>
          {emission.category}
        </span>
      </header>

      <p className="text-3xl font-semibold tracking-tight text-slate-900">
        <span aria-hidden="true">
          {formatCo2(emission.co2Kg)}
          <span className="ml-1 text-sm font-normal text-slate-600">CO₂e</span>
        </span>
        <span className="sr-only">{formatCo2Spoken(emission.co2Kg)}</span>
      </p>

      <time dateTime={emission.measuredAt} className="mt-auto text-xs text-slate-600">
        {formatDate(emission.measuredAt)}
      </time>
    </article>
  )
})
```

src/components/EmissionsStatus.tsx

```
interface LoadErrorProps {
  readonly message: string
  readonly onRetry: () => void
}

/** Esqueleto de carga. <output> tiene role="status" implícito y se anuncia de forma educada. */
export function EmissionsLoading() {
  return (
    <output aria-live="polite" className="block">
      <span className="sr-only">Cargando mediciones…</span>
      <ul aria-hidden="true" className="grid gap-4 sm:grid-cols-2 lg:grid-cols-3">
```

```
        {[0, 1, 2].map((i) => (
          <li key={i} className="flex animate-pulse flex-col motion-reduce:animate-none gap-3 rounded-xl border border-slate-200 bg-white
p-5">
            <div className="h-4 w-2/3 rounded bg-slate-200" />
            <div className="h-8 w-1/2 rounded bg-slate-200" />
            <div className="h-3 w-1/3 rounded bg-slate-100" />
          </li>
        ))}
      </ul>
    </output>
  )
}

/** Error de carga con reintento. role="alert" lo anuncia apenas aparece. */
export function EmissionsLoadError({ message, onRetry }: LoadErrorProps) {
  return (
    <div role="alert" className="flex flex-col items-center gap-3 rounded-xl border border-red-200 bg-red-50 p-6 text-center">
      <p className="text-sm font-medium text-red-800">No se pudieron cargar las mediciones.</p>
      <p className="text-sm text-red-700">{message}</p>
      <button
        type="button"
        onClick={onRetry}
        className="rounded-lg border border-red-300 bg-white px-4 py-2 text-sm font-medium text-red-800 hover:bg-red-100 focus-
visible:outline-2 focus-visible:outline-offset-2 focus-visible:outline-red-700"
      >
        Reintentar
      </button>
    </div>
  )
}

export function EmissionsEmpty() {
  return (
    <p className="rounded-xl border border-dashed border-slate-300 bg-white p-10 text-center text-sm text-slate-600">
      Todavía no hay mediciones. Agrega la primera con el formulario.
    </p>
  )
}
```

src/components/EmissionsSummary.tsx

```
import { memo } from 'react'
import { EMISSION_CATEGORIES } from '../types/emissions.ts'
import type { EmissionsSummary as Summary } from '../types/emissions.ts'
import { formatCo2, formatCo2Spoken } from '../utils/format.ts'

interface EmissionsSummaryProps {
  readonly summary: Summary
  /** Mientras carga se muestra «—»: un «0 kg» haría creer que la empresa no emite nada. */
  readonly loading: boolean
}

/** Total y desglose por categoría. Recibe el resumen ya memorizado por el contenedor. */
export const EmissionsSummary = memo(function EmissionsSummary({ summary, loading }: EmissionsSummaryProps) {
  const value = (kg: number) =>
    loading ? (
      <>
        <span aria-hidden="true">—</span>
        <span className="sr-only">cargando</span>
      </>
    ) : (
      <>
        <span aria-hidden="true">{formatCo2(kg)}</span>
        <span className="sr-only">{formatCo2Spoken(kg)}</span>
      </>
    )

  return (
    <dl aria-busy={loading} className="grid grid-cols-2 gap-3 sm:grid-cols-4">
      <div className="col-span-2 rounded-xl bg-slate-900 p-4 text-white sm:col-span-1">
        <dt className="text-xs font-medium tracking-wide text-slate-300 uppercase">Total</dt>
        <dd className="mt-1 text-2xl font-semibold">
          {value(summary.totalKg)}
          {!loading && <span aria-hidden="true"> CO₂e</span>}
        </dd>
        <dd className="text-xs text-slate-300">
          {loading ? 'Cargando…' : `${summary.count} ${summary.count === 1 ? 'fuente registrada' : 'fuentes registradas'}`}
        </dd>
      </div>
      {EMISSION_CATEGORIES.map((category) => (
        <div key={category} className="rounded-xl border border-slate-200 bg-white p-4">
          <dt className="text-xs font-medium tracking-wide text-slate-600 uppercase">{category}</dt>
          <dd className="mt-1 text-lg font-semibold text-slate-900">
            {value(summary.byCategory[category])}
          </dd>
        </div>
      ))}
    </dl>
  )
})
```

supabase/schema.sql

```sql
-- EcoTrack · tabla de mediciones de emisiones (v4, auditada)
-- Ejecutar en el SQL Editor de Supabase o como migración. Es idempotente: se puede correr dos veces.
-- Pruebas: bash herramientas/probar-rls.sh  (PostgreSQL 17 en Docker, rol anon, 9 pruebas)

create table if not exists public.emissions (
  id          uuid primary key default gen_random_uuid(),
  source_name text not null,
  category    text not null,
  co2_kg      numeric(12, 2) not null,
  measured_at timestamptz not null default now(),
  -- Las mismas reglas que valida el formulario (src/utils/validation.ts), repetidas en la BD:
  -- el frontend no es una frontera de seguridad.
  constraint emissions_source_name_len check (char_length(source_name) between 1 and 80 and source_name = btrim(source_name)),
  constraint emissions_category_valid  check (category in ('Transporte', 'Energía', 'Residuos')),
  constraint emissions_co2_range       check (co2_kg >= 0 and co2_kg <= 9999999999.99)
);

create index if not exists emissions_measured_at_idx on public.emissions (measured_at desc);

-- 1. Privilegios: QUÉ operaciones puede hacer cada rol.
-- En Supabase cada tabla nueva de public nace con TODO concedido a anon y authenticated
-- (incluido truncate, que RLS no filtra). Las políticas no quitan esos privilegios: se revocan.
revoke all on table public.emissions from public, anon, authenticated;
grant select on table public.emissions to anon, authenticated;
-- Insert solo de lo que aporta el usuario: id y measured_at los pone la BD y no se pueden falsificar.
grant insert (source_name, category, co2_kg) on table public.emissions to anon, authenticated;
-- Sin update, delete ni truncate: una medición registrada no se edita desde el frontend.

-- 2. Row Level Security: SOBRE QUÉ filas.
alter table public.emissions enable row level security;

drop policy if exists "emissions_select_todos" on public.emissions;
create policy "emissions_select_todos"
  on public.emissions for select
  to anon, authenticated
  using (true);

drop policy if exists "emissions_insert_valida" on public.emissions;
create policy "emissions_insert_valida"
  on public.emissions for insert
  to anon, authenticated
  with check (
    category in ('Transporte', 'Energía', 'Residuos')
    and co2_kg between 0 and 9999999999.99
    and char_length(btrim(source_name)) between 1 and 80
  );

-- En producción: quitar anon de las políticas, exigir sesión (authenticated) y agregar una columna
-- organization_id con políticas del tipo
--   using (organization_id = (select organization_id from public.members where user_id = (select auth.uid())))
-- para que cada empresa vea e inserte solo sus mediciones. Este lab no tiene login, por eso anon lee e inserta.
```

.env.example

```
# Copia este archivo como .env y llena los valores de tu proyecto de Supabase.
# .env NO se sube al repositorio (está en .gitignore).
# Solo la anon key: la service_role nunca va en el frontend ni lleva el prefijo VITE_.
VITE_SUPABASE_URL=
VITE_SUPABASE_ANON_KEY=
```

src/hooks/useEmissions.test.ts

```ts
import { act, renderHook, waitFor } from '@testing-library/react'
import { describe, expect, it, vi } from 'vitest'
import type { Emission, EmissionsRepository, NewEmission } from '../types/emissions.ts'
import { useEmissions } from './useEmissions.ts'

const SAMPLE: Emission = {
  id: 'e-1',
  sourceName: 'Caldera planta norte',
  category: 'Energía',
  co2Kg: 120,
  measuredAt: '2026-11-03T10:00:00Z',
}

const NEW: NewEmission = { sourceName: 'Camión 12', category: 'Transporte', co2Kg: 45.5 }

/** Repositorio falso: cada prueba decide qué devuelve o qué error lanza. */
function fakeRepository(overrides: Partial<EmissionsRepository> = {}): EmissionsRepository {
  return {
    mode: 'demo',
    simulation: null,
    list: async () => [SAMPLE],
    insert: async (input) => ({ ...input, id: 'e-2', measuredAt: '2026-11-03T11:00:00Z' }),
    ...overrides,
  }
}

/** Promesa que la prueba resuelve cuando quiere (para observar estados intermedios). */
function deferred<T>() {
  let resolve: (value: T) => void = () => {}
  const promise = new Promise<T>((r) => {
```

```
    resolve = r
  })
  return { promise, resolve }
}

describe('useEmissions', () => {
  it('carga el listado: loading y después success', async () => {
    const { result } = renderHook(() => useEmissions(fakeRepository()))
    expect(result.current.status).toBe('loading')

    await waitFor(() => expect(result.current.status).toBe('success'))
    expect(result.current.emissions).toEqual([SAMPLE])
    expect(result.current.loadError).toBeNull()
  })

  it('regresión v2: un repositorio nuevo en cada render no provoca recargas sin fin', async () => {
    const list = vi.fn(async () => [SAMPLE])
    const { result, rerender } = renderHook(() => useEmissions(fakeRepository({ list })))
    await waitFor(() => expect(result.current.status).toBe('success'))
    rerender()
    rerender()
    expect(list).toHaveBeenCalledTimes(1)
  })

  it('si la carga falla expone el mensaje; reload vuelve a intentar', async () => {
    let attempts = 0
    const repo = fakeRepository({
      list: async () => {
        attempts++
        if (attempts === 1) throw new Error('No se pudo conectar con Supabase.')
        return [SAMPLE]
      },
    })
    const { result } = renderHook(() => useEmissions(repo))
    await waitFor(() => expect(result.current.status).toBe('error'))
    expect(result.current.loadError).toBe('No se pudo conectar con Supabase.')

    act(() => result.current.reload())
    expect(result.current.status).toBe('loading')
    await waitFor(() => expect(result.current.status).toBe('success'))
    expect(result.current.loadError).toBeNull()
    expect(result.current.emissions).toEqual([SAMPLE])
  })

  it('inserta y pone la nueva medición de primera', async () => {
    const { result } = renderHook(() => useEmissions(fakeRepository()))
    await waitFor(() => expect(result.current.status).toBe('success'))

    let ok = false
    await act(async () => {
      ok = await result.current.addEmission(NEW)
    })

    expect(ok).toBe(true)
    expect(result.current.emissions.map((e) => e.id)).toEqual(['e-2', 'e-1'])
    expect(result.current.isSubmitting).toBe(false)
  })

  it('si la inserción falla conserva la lista y expone submitError', async () => {
    const repo = fakeRepository({
      insert: async () => {
        throw new Error('No tienes permiso para esta operación (política RLS).')
      },
    })
    const { result } = renderHook(() => useEmissions(repo))
    await waitFor(() => expect(result.current.status).toBe('success'))

    let ok = true
    await act(async () => {
      ok = await result.current.addEmission(NEW)
    })

    expect(ok).toBe(false)
    expect(result.current.submitError).toBe('No tienes permiso para esta operación (política RLS).')
    expect(result.current.emissions).toEqual([SAMPLE])
    expect(result.current.isSubmitting).toBe(false)

    act(() => result.current.clearSubmitError())
    expect(result.current.submitError).toBeNull()
  })

  it('bloquea el doble envío mientras hay una inserción en curso', async () => {
    const pending = deferred<Emission>()
    const insert = vi.fn(() => pending.promise)
    const { result } = renderHook(() => useEmissions(fakeRepository({ insert })))
    await waitFor(() => expect(result.current.status).toBe('success'))

    let first: Promise<boolean> = Promise.resolve(false)
    let second = true
    await act(async () => {
      first = result.current.addEmission(NEW)
      second = await result.current.addEmission(NEW)
    })
    expect(second).toBe(false)
    expect(insert).toHaveBeenCalledTimes(1)
    expect(result.current.isSubmitting).toBe(true)

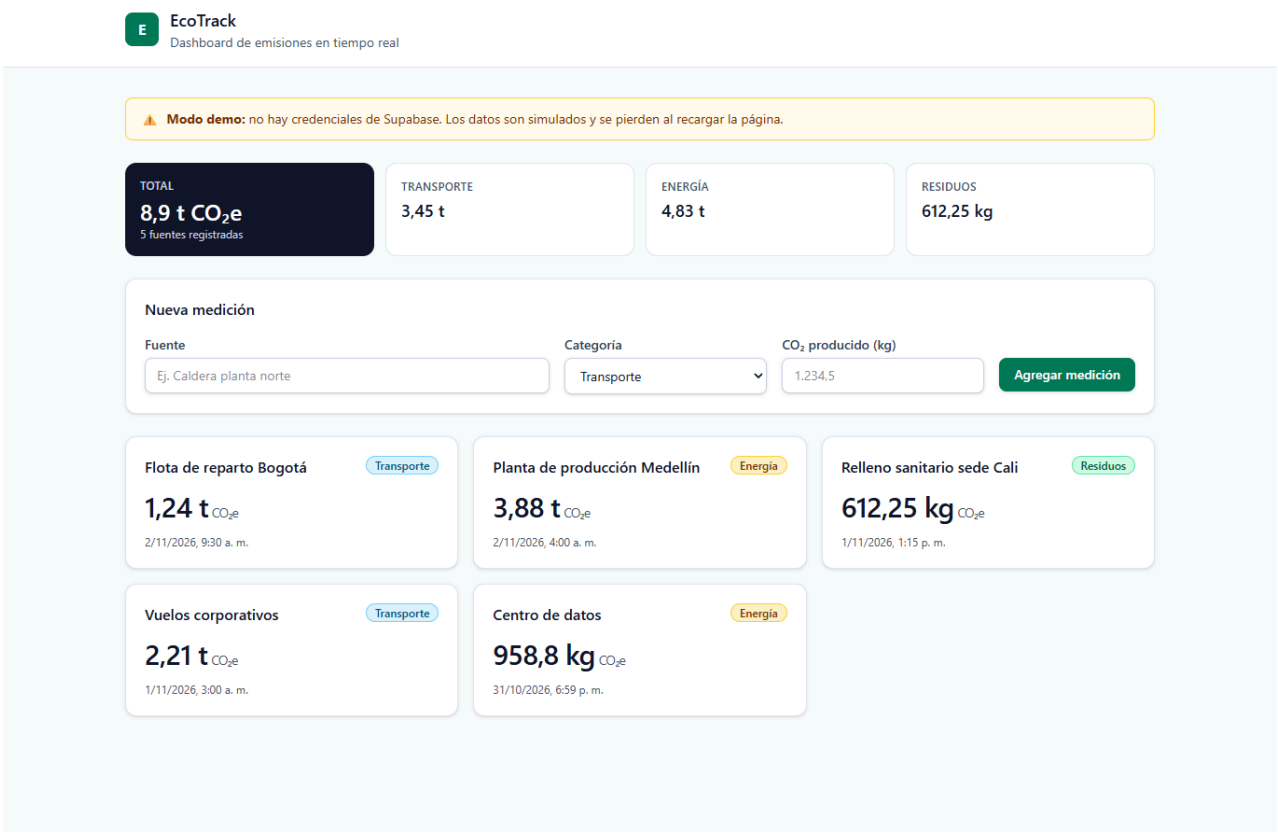
    await act(async () => {
```

```
      pending.resolve({ ...NEW, id: 'e-9', measuredAt: '2026-11-03T12:00:00Z' })
      await first
    })
    expect(result.current.isSubmitting).toBe(false)
    expect(result.current.emissions[0]?.id).toBe('e-9')
  })

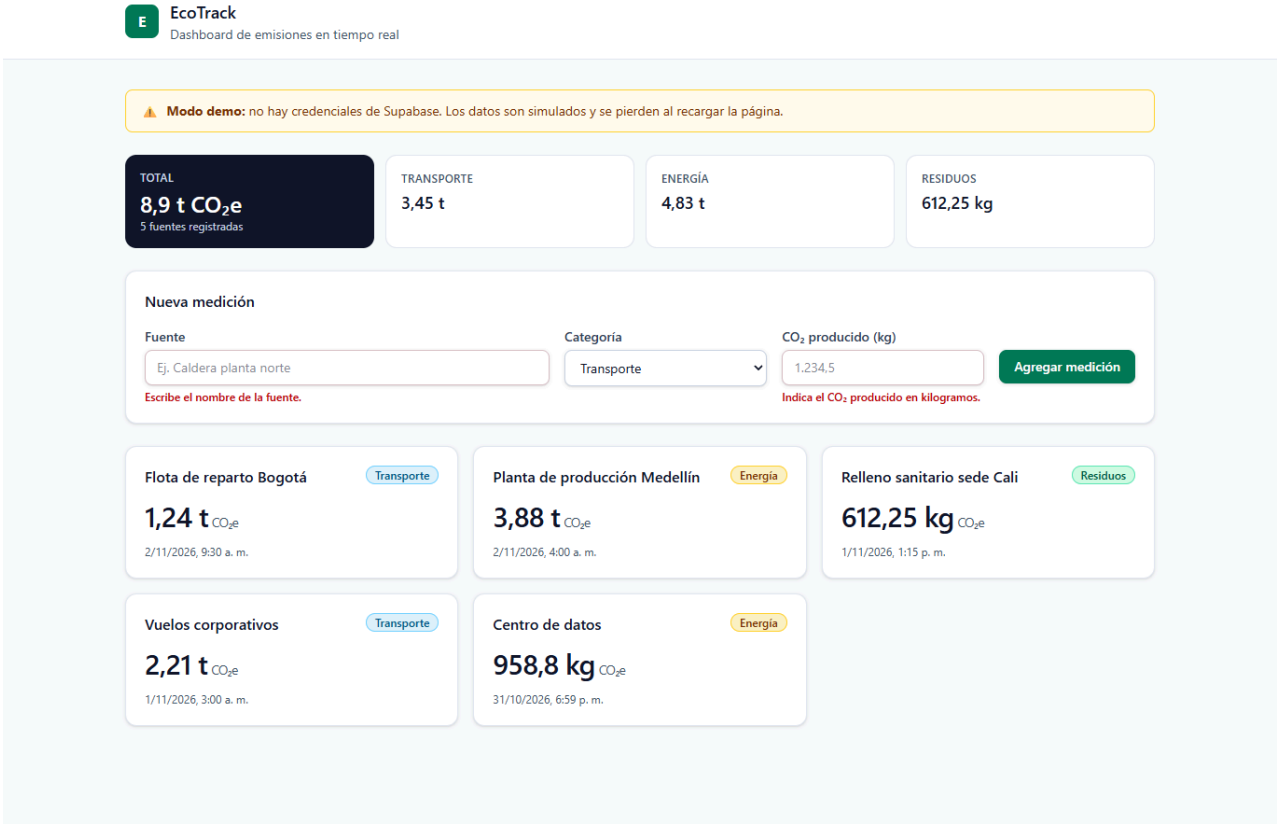
  it('el objeto devuelto es estable entre renders si nada cambió (useMemo)', async () => {
    const { result, rerender } = renderHook(() => useEmissions(fakeRepository()))
    await waitFor(() => expect(result.current.status).toBe('success'))
    const before = result.current
    rerender()
    expect(result.current).toBe(before)
  })
})
```

Parte 4 — Capturas de la interfaz funcionando

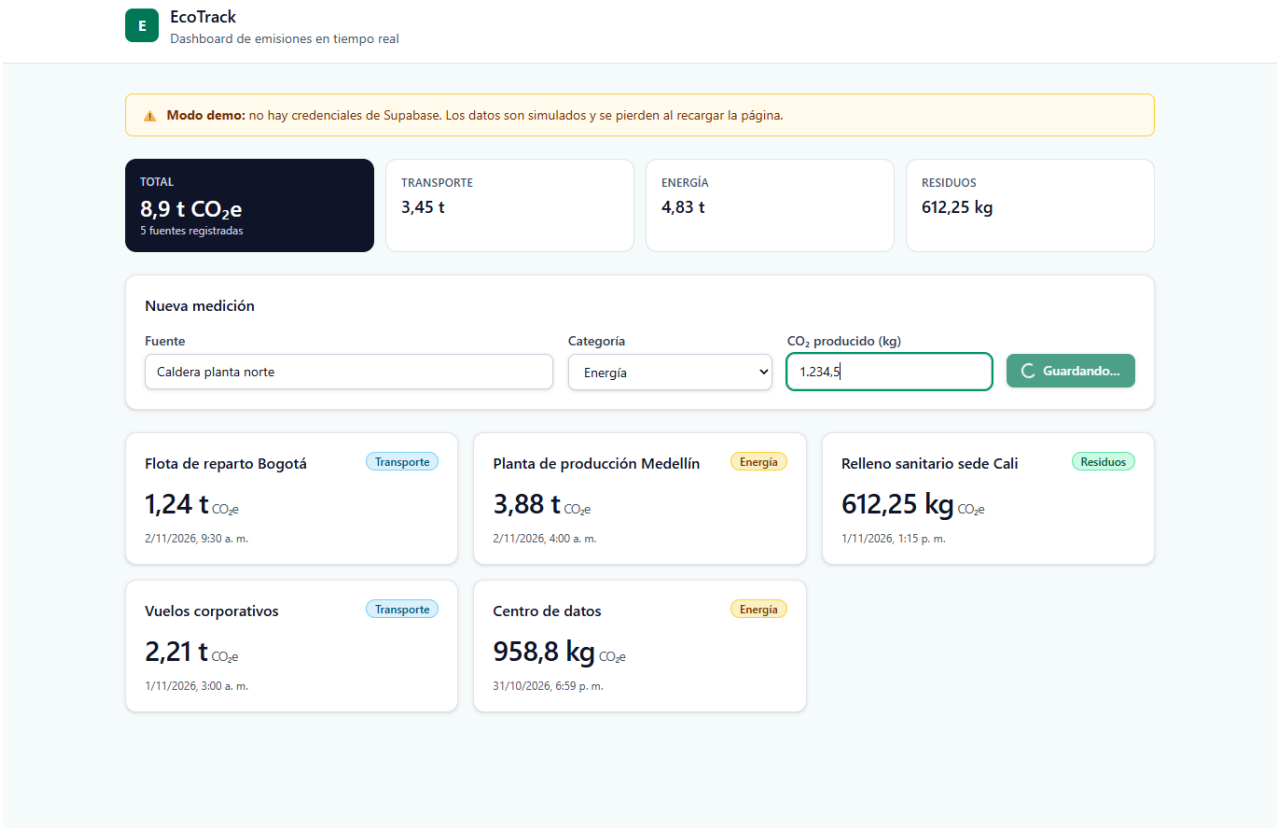
01-escritorio-lista



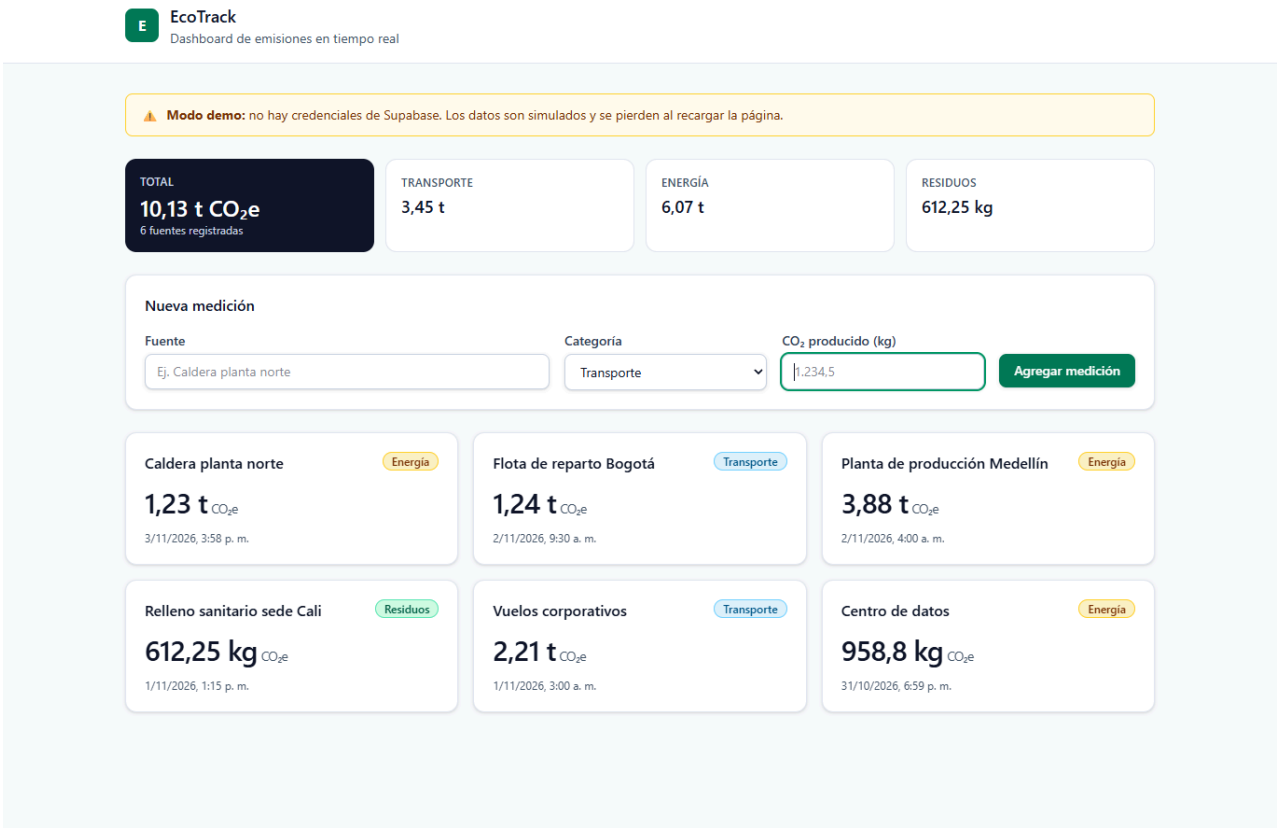
02-escritorio-validacion



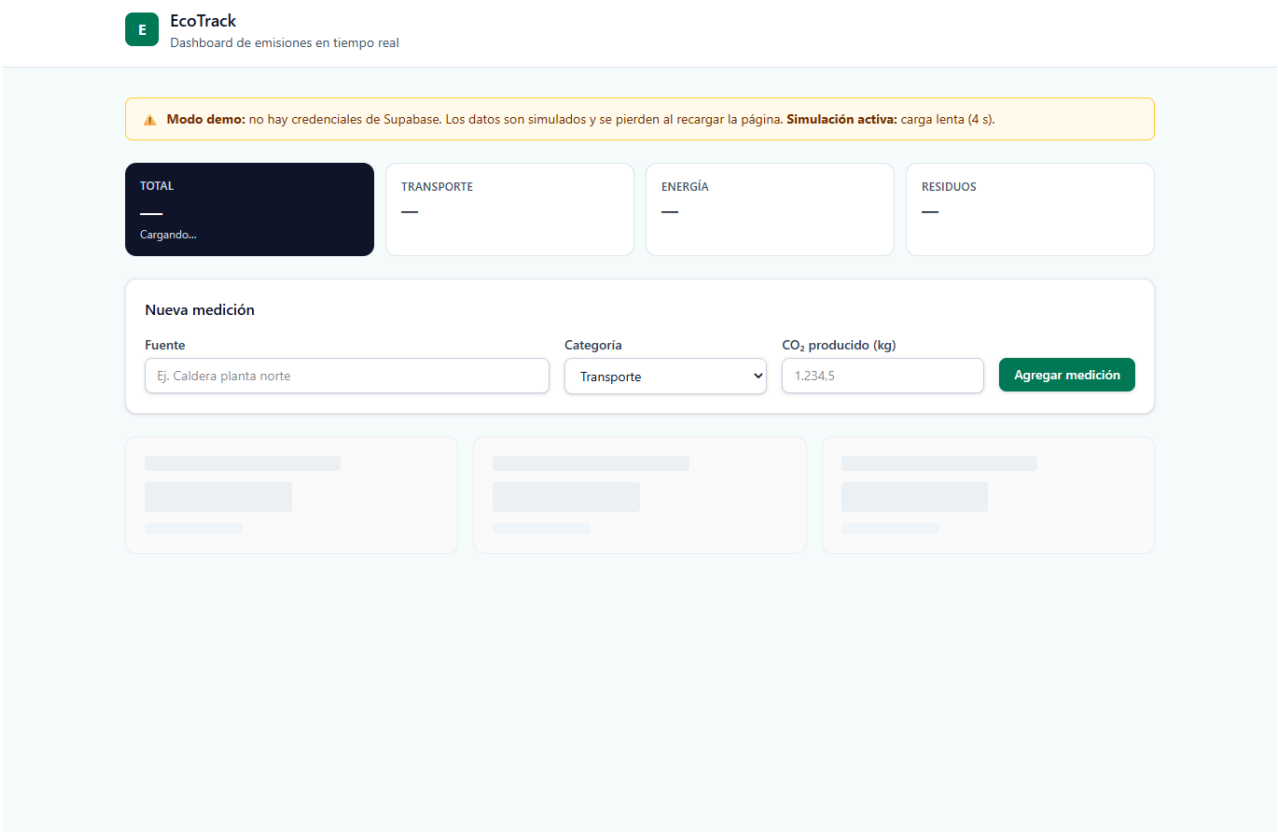
03-escritorio-guardando



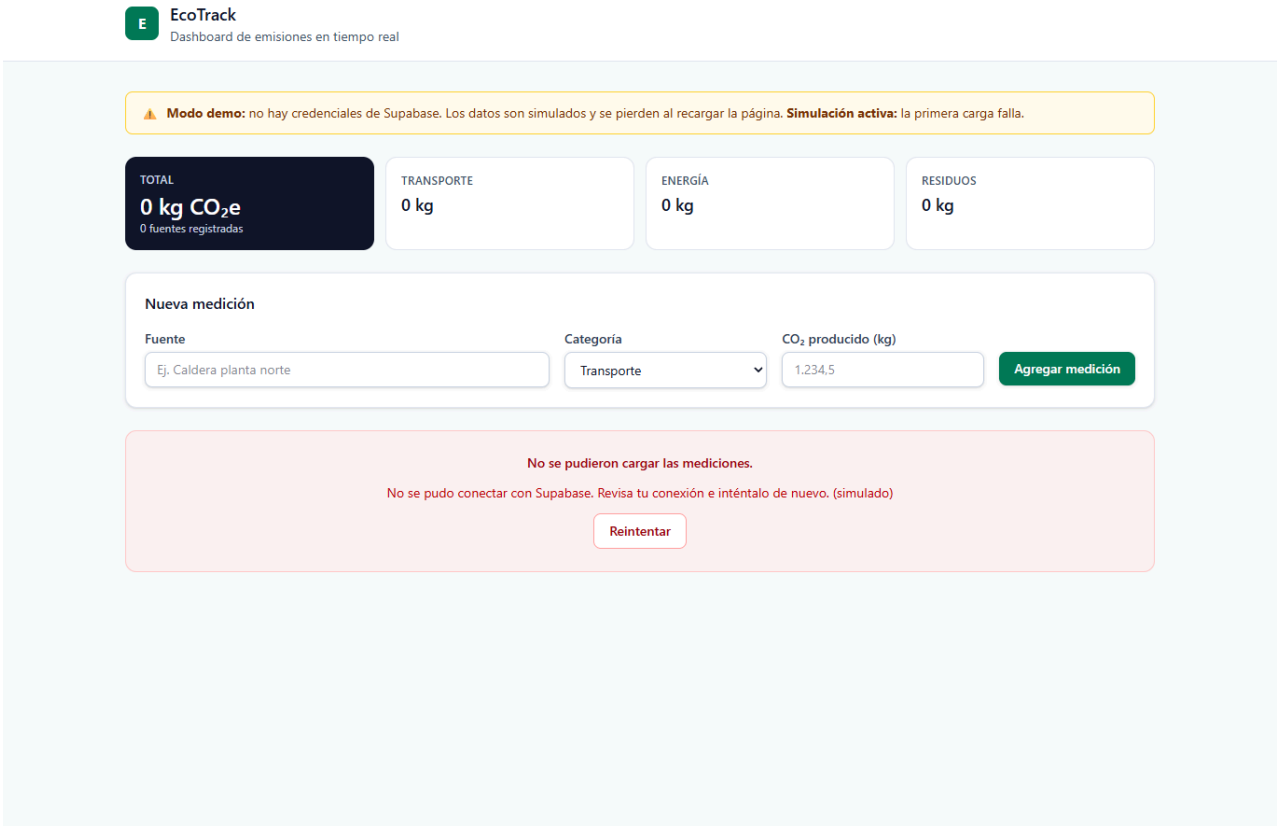
04-escritorio-medicion-agregada



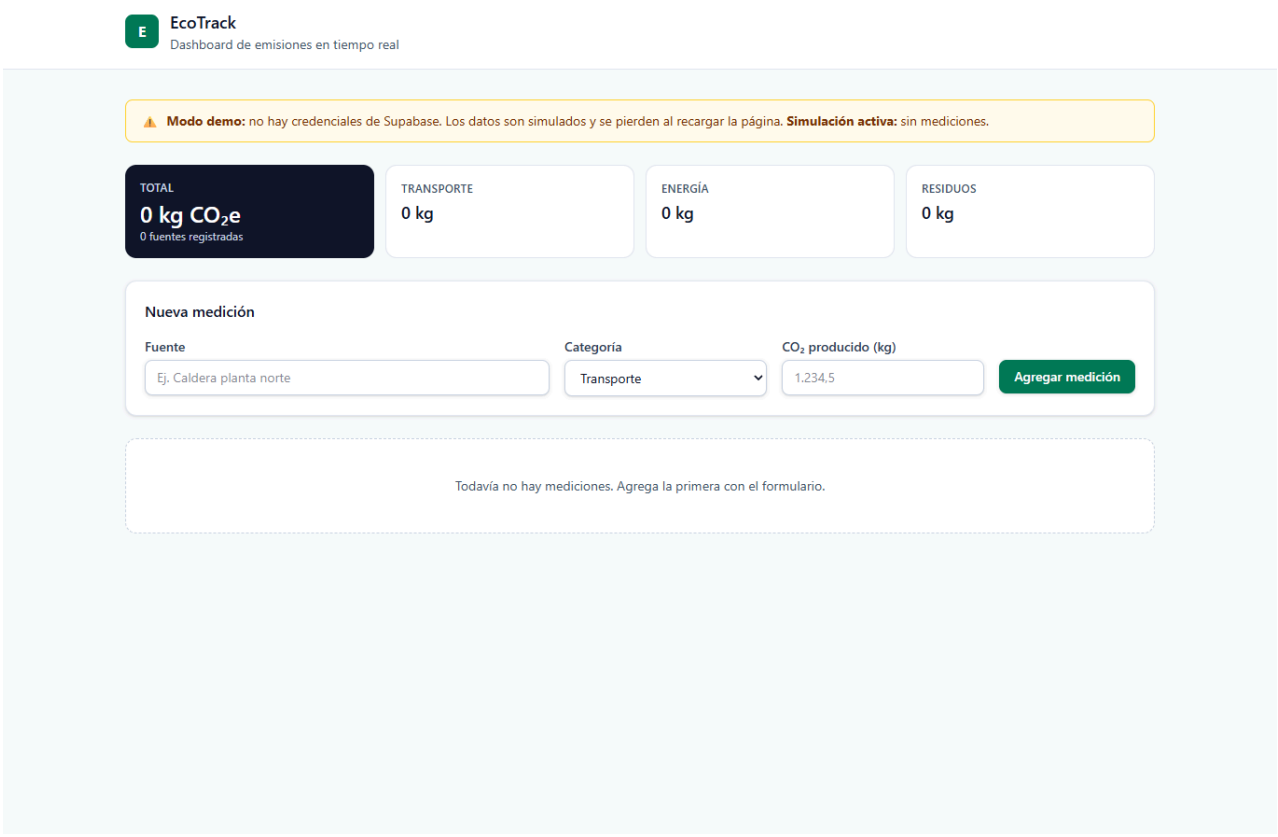
05-carga-esqueleto



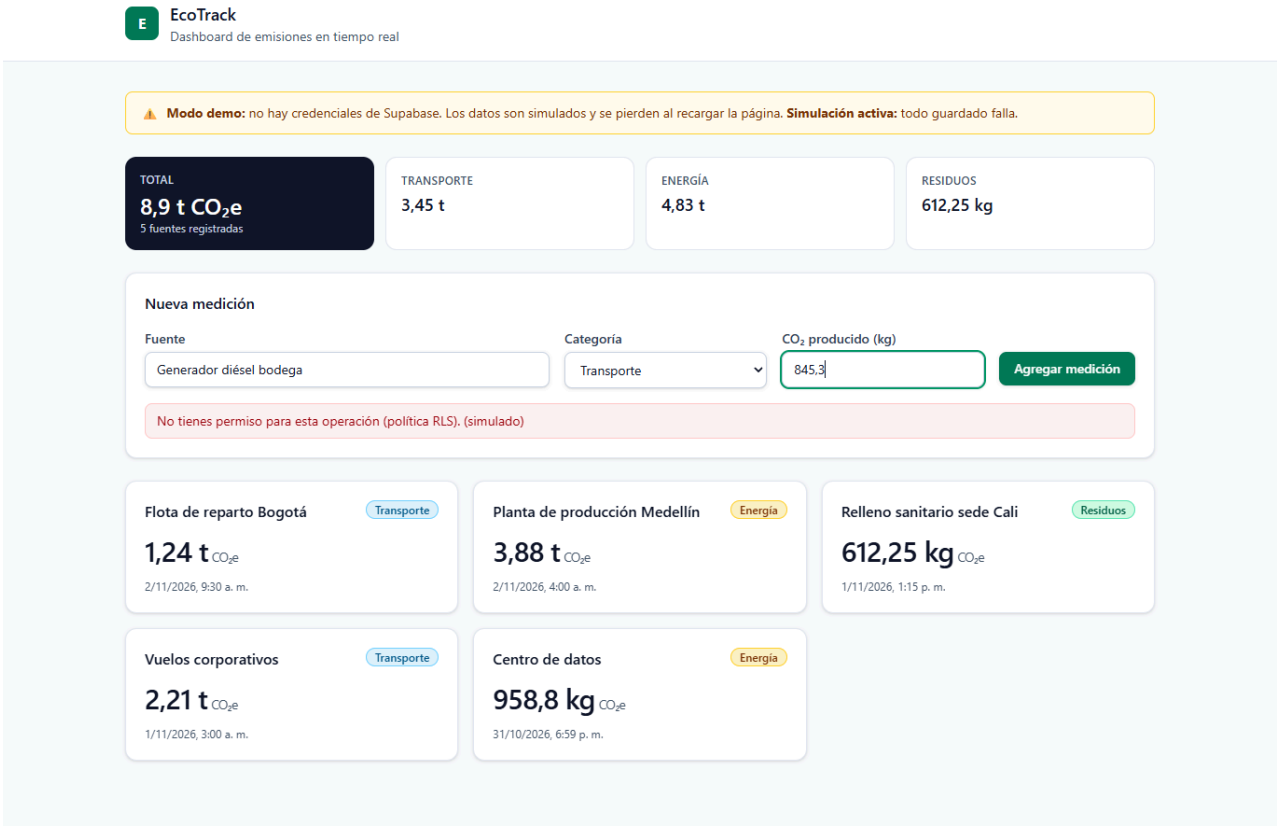
06-error-de-carga



07-lista-vacia



08-error-al-guardar



09-movil-inicio



10-movil-cards



11-movil-validacion

Nueva medición

Fuente

Ej. Caldera planta norte

Escribe el nombre de la fuente.

Categoría

Transporte

CO_2 producido (kg)

1.234,5

Indica el CO_2 producido en kilogramos.

Agregar medición

Flota de reparto Bogotá

Transporte

1,24 t CO_2e

2/11/2026, 9:30 a. m.

Planta de producción Medellín

Energía

3,88 t CO_2e

2/11/2026, 4:00 a. m.

Relleno sanitario sede Cali

Residuos

612,25 kg CO_2e

Parte 5 — Verificación final (salida real)

```
Verificación final de src/ · 2026-11-03 16:04:04 -0500 · node v24.15.0 · pnpm 11.24.0 · commit e78f5ef

### $ pnpm install --frozen-lockfile
Already up to date
Done in 2s using pnpm v11.24.0
(código de salida: 0)

Nota: tsconfig.json tiene "files": [] y solo referencias, así que `pnpm tsc --noEmit` no revisa ningún
archivo y sale con 0 aunque haya errores. El chequeo real es `tsc -b` (script typecheck).

### $ pnpm tsc --noEmit
(código de salida: 0)

### $ pnpm typecheck
$ tsc -b --noEmit
(código de salida: 0)

### $ pnpm lint
$ oxlint
(código de salida: 0)

### $ pnpm test
$ vitest run

 RUN  v5.1.0 C:/Users/josue/OneDrive/Documentos/Repositorios/universidad/ecotrack-emissions-tracker


 Test Files  3 passed (3)
      Tests  37 passed (37)
   Start at  16:04:41
   Duration  9.19s (environment 81%, import 12%, tests 5%, transform 1%, worker 1%)

(código de salida: 0)

### $ pnpm build
$ tsc -b && vite build
vite v8.3.5 building client environment for production...
transforming...
✓ 72 modules transformed.
rendering chunks...
computing gzip size...
dist/index.html                   0.43 kB │ gzip:  0.30 kB
dist/assets/index-CQIYEn4L.css   18.53 kB │ gzip:  4.63 kB
dist/assets/index-CBPE1vMD.js   233.20 kB │ gzip: 73.15 kB

✓ built in 1.19s
(código de salida: 0)

### Build con variables de prueba (valores falsos, no son credenciales): tamaño real del bundle
### $ pnpm exec vite build
vite v8.3.5 building client environment for production...
transforming...
✓ 72 modules transformed.
rendering chunks...
computing gzip size...
dist/index.html                   0.43 kB │ gzip:   0.30 kB
dist/assets/index-CQIYEn4L.css   18.53 kB │ gzip:   4.63 kB
dist/assets/index-DNptQ5a1.js   449.37 kB │ gzip: 128.33 kB

✓ built in 1.33s
(código de salida: 0)

### RLS de supabase/schema.sql (PostgreSQL 17 en Docker, rol anon)
### $ bash herramientas/probar-rls.sh
== 2026-11-03 16:05:41 -0500 · PostgreSQL 17.9 on x86_64-pc-linux-musl, compiled by gcc (Alpine 15.2.0) 15.2.0, 64-bit
== Schema: supabase/schema.sql
NOTICE:  policy "emissions_select_todos" for relation "public.emissions" does not exist, skipping
NOTICE:  policy "emissions_insert_valida" for relation "public.emissions" does not exist, skipping
== Schema aplicado por segunda vez (idempotencia)
NOTICE:  relation "emissions" already exists, skipping
NOTICE:  relation "emissions_measured_at_idx" already exists, skipping
OK    1. anon puede leer las mediciones
OK    2. anon inserta una medición válida
OK    3. categoría fuera de la lista rechazada (política RLS)
OK    4. co2_kg negativo rechazado (política RLS)
OK    5. el cliente no puede fijar measured_at
OK    6. el cliente no puede elegir el id
OK    7. update denegado
OK    8. delete denegado
OK    9. truncate denegado
       ?column?
----------------------
 filas al terminar: 2
(1 row)

(código de salida: 0)
```